

Лабораторной работе №1

Подготовка стенда

Еремина Оксана Андреевна

Содержание

1	Введение	5
1.1	Цель работы	5
1.2	Задачи	5
1.3	Объект и предмет исследования	6
1.4	Методы исследования	6
2	Теоретическая часть	7
3	Материалы и методы	8
4	Выполнение лабораторной работы	10
4.1	Настройка среды	10
4.2	Выполнение примеров (раздел 1.3.3)	11
4.3	Выполнение заданий	12
5	Результаты	17
6	Обсуждение результатов	18
7	Заключение	19
	Список литературы	20

Список иллюстраций

4.1	Установка Julia	10
4.2	JupyterLab	11
4.3	Функции ввода/вывода	13
4.4	Математические операции	15
4.5	Операции с матрицами и векторами	16

Список таблиц

1 Введение

1.1 Цель работы

Основная цель работы — подготовить рабочее пространство и инструментарий для работы с языком программирования Julia, на простейших примерах познакомиться с основами синтаксиса Julia.

1.2 Задачи

1. Установить Julia с помощью пакетного менеджера.
2. Установить Jupyter Lab и пакет IJulia.
3. Запустить Jupyter Lab и создать новый ноутбук с ядром Julia.
4. Выполнить примеры из раздела 1.3.3 методических указаний.
5. Самостоятельно выполнить задания 1–4 по вводу/выводу, парсингу, математическим операциям и операциям с матрицами.

1.3 Объект и предмет исследования

Объект: среда выполнения кода Julia (Jupyter Lab, REPL).

Предмет: базовые синтаксические конструкции языка Julia и работа с пакетами.

1.4 Методы исследования

В работе использованы следующие методы:

- анализ документации языка Julia и Jupyter;
- установка программного обеспечения с помощью менеджеров пакетов (snap, Pkg);
- экспериментальное выполнение программного кода в интерактивной среде Jupyter Lab;

2 Теоретическая часть

Julia — высокоуровневый язык программирования с динамической типизацией, разработанный для научных вычислений [2]. Он сочетает скорость выполнения, близкую к C, и простоту синтаксиса, как у Python или MATLAB.

Работа с командной строкой и пакетными менеджерами подробно описана в литературе [4—6].

Ключевые особенности:

- Компиляция JIT (Just-In-Time) на основе LLVM.
- Встроенная поддержка многопоточности и распределённых вычислений.
- Богатая экосистема пакетов (DataFrames, Plots, LinearAlgebra, StatsBase и др.).
- Возможность работы в Jupyter через ядро IJulia [1].

Для управления пакетами используется встроенный менеджер Pkg. Установка пакета выполняется командой `Pkg.add(«ИмяПакета»)`.

Jupyter Lab — интерактивная веб-среда для работы с ноутбуками, поддерживающая множество языков (Python, Julia, R и др.) [3].

3 Материалы и методы

Оборудование:

- Виртуальная машина на базе Apple Virtualization (Ubuntu Linux).

Программное обеспечение:

- ОС: Ubuntu (версия Resolute).
- Пакетные менеджеры: apt, snap.
- Интерпретатор Julia версии 1.12.5.
- Jupyter Lab версии 4.6.3.
- Браузер для доступа к Jupyter Lab.

Использование операционной системы Linux и её стандартных утилит рассматривается в [7].

Методы:

- Установка Julia через snap.
- Установка IJulia через встроенный менеджер пакетов Julia Pkg.

- Запуск Jupyter Lab из терминала.
- Выполнение кода в ячейках ноутбука.

4 Выполнение лабораторной работы

4.1 Настройка среды

Julia была успешно установлена через Curl и после установки проверена версия

```
ksusha@oaeremtna:~$ curl -fsSL https://install.julialang.org | sh
info: downloading installer
Welcome to Julia!

This will download and install the official Julia Language distribution
and its version manager Juliaup.

Juliaup will be installed into the Juliaup home directory, located at:

    /home/ksusha/.juliaup

The julia, juliaup and other commands will be added to
Juliaup's bin directory, located at:

    /home/ksusha/.juliaup/bin

This path will then be added to your PATH environment variable by
modifying the profile files located at:

    /home/ksusha/.bashrc
    /home/ksusha/.profile
```

Рисунок 4.1: Установка Julia

После запуска в браузере открылся интерфейс JupyterLab. В панели Launcher доступны ядра Python 3 и Julia

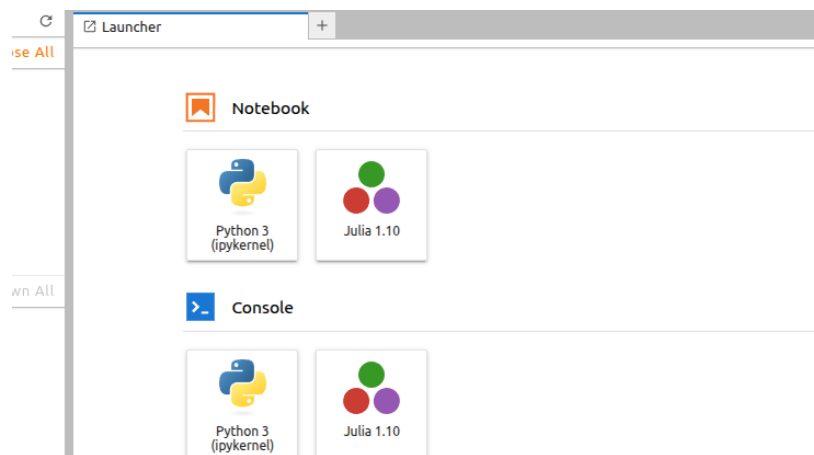


Рисунок 4.2: JupyterLab

4.2 Выполнение примеров (раздел 1.3.3)

В ноутбуке выполнены примеры: - арифметические операции ($2+3$, $3+4$, $1+2$) - подавление вывода с помощью ; - определение типов числовых величин (typeof) - вывод диапазонов целочисленных типов - справка по функции println (?println) - команда ;pwd для вывода текущей директории

```
Launcher x lab01.ipynb x +
Notebook Julia 1.10

[1]: ## Простейшие арифметические операции

[2]: 2+3

[2]: 5

[3]: 3+4
1+2

[3]: 3

[4]: typeof(3), typeof(3.5), typeof(3/3.5), typeof(sqrt(3+4im)), typeof(pi)

[4]: (Int64, Float64, Float64, ComplexF64, Irrational{::N})

[5]: 1.0/0.0, 1.0/(-0.0), 0.0/0.0

[5]: (Inf, -Inf, NaN)

[6]: for T in [Int8, Int16, Int32, Int64, Int128, UInt8, UInt16, UInt32, UInt64, UInt128]
      println("%${pad(T,7)}: ${typemin(T)}, ${typemax(T)}")
    end

      Int8: [-128,127]
      Int16: [-32768,32767]
      Int32: [-2147483648,2147483647]
      Int64: [-9223372036854775808,9223372036854775807]
      Int128: [-170141183460469231731687303715884105728,170141183460469231731687303715884105727]
      UInt8: [0,255]
      UInt16: [0,65535]
      UInt32: [0,4294967295]
      UInt64: [0,18446744073709551615]
      UInt128: [0,340282366920938463463374607431768211455]

[7]: Int64(2.0), Char(2), typeof(Char(2))
      convert(Int64, 2.0), convert(Char,2)
      typeof(promote(Int8(1), Float16(4.5), Float32(4.1)))

[7]: Tuple{Float32, Float32, Float32}

[13]: ?println

      search: println printstyled print sprint isprint

[13]: println([io::IO], xs...)
      Print (using print) xs to io followed by a newline. If io is not supplied, prints to the default output
      stream stdout.

      See also printstyled to add colors etc.

Examples

julia> println("Hello, world")
Hello, world

julia> io = IOBuffer();

julia> println(io, "Hello", ',', " world.")

julia> String(take!(io))
"Hello, world.\n"

[14]: ;date
      ;whoami

      UndefVarError: `date` not defined
```

```
[8]: function f(x)
      x^2
    end
      f(4)

[8]: 16

[9]: g(x) = x^2
      g(8)

[9]: 64

[10]: a = [4 7 6]
      b = [1, 2, 3]
      a[2], b[2]

[10]: (7, 2)

[11]: a = 1; b = 2; c = 3; d = 4
      Am = [a b; c d]
      Am[1,1], Am[1,2], Am[2,1], Am[2,2]

[11]: (1, 2, 3, 4)

[12]: aa = [1 2]
      AA = [1 2; 3 4]
      aa*AA*aa'

[12]: 1x1 Matrix{Int64}:
      27
```

4.3 Выполнение заданий

4.3.1 Задание 1. Функции ввода/вывода

Выполнен код:

- `print`, `println`, `show`;
- побайтовую запись через `write` в `IOBuffer`;
- запись и чтение файлов (`write`, `read`, `readline`, `readlines`);
- чтение табличных данных через `readlrm` (пакет `DelimitedFiles`).

Вывод программы показывает корректное выполнение всех операций

```
[15]: ## Задания для самостоятельной работы

[16]: print("Hello")           # без перевода строки
      print(" ")
      print("world")
      println()               # просто перевод строки

      println("Hello, world") # с переводом строки
      println("a = ", 5, ", b = ", 3.14)

      Hello world
      Hello, world
      a = 5, b = 3.14

[17]: show(3.14); println()
      show(stdout, "text/plain", [1 2; 3 4])

      3.14
      2x2 Matrix{Int64}:
      1  2
      3  4

[18]: io = IOBuffer()
      n = write(io, "Hello")
      println("записано байт: ", n)
      String(take!(io))      # → "Hello"

      записано байт: 5
[18]: "Hello"

[19]: # Запись
      open("test.txt", "w") do f
          write(f, "строка 1\nстрока 2\nстрока 3\n")
      end

      # read – чтение всего файла как строки
      content = read("test.txt", String)
      println(content)

      строка 1
      строка 2
      строка 3
```

Рисунок 4.3: Функции ввода/вывода

4.3.2 Задание 2. Функции `parse()`

- преобразование строки в `Int64` (`parse{Int, «123»`);
- преобразование в `Float64` (`parse{Float64, «3.14»`);
- преобразование строки в `Bool`;
- парсинг числа в заданной системе счисления (`base=16`);
- генерация ошибки при попытке парсинга нечисловой строки и безопасный вариант через `tryparse`.

```
[20]: open("test.txt") do f
      line1 = readline(f)
      line2 = readline(f)
      println("1-я строка: ", line1)
      println("2-я строка: ", line2)
    end

1-я строка: строка 1
2-я строка: строка 2

[21]: lines = readlines("test.txt")
      println(lines)
      println("тип: ", typeof(lines))

["строка 1", "строка 2", "строка 3"]
тип: Vector{String}

[22]: using DelimitedFiles

# Записать CSV
writedlm("data.csv", [1 2 3; 4 5 6; 7 8 9], ',')

# Прочитать CSV
M = readlm("data.csv", ',')
println(M)
println("тип: ", typeof(M))
println("размер: ", size(M))

[1.0 2.0 3.0; 4.0 5.0 6.0; 7.0 8.0 9.0]
тип: Matrix{Float64}
размер: (3, 3)

[23]: # 1. Строка → целое
      parse{Int, "123"}          # → 123
      parse{Int64, "42"}         # → 42

      # 2. Строка → вещественное
      parse{Float64, "3.14"}      # → 3.14
      parse{Float32, "2.71"}      # → 2.71f0

      # 3. Строка → Bool
      parse{Bool, "true"}         # → true
      parse{Bool, "false"}        # → false
```

```
# 4. Система счисления
parse{Int, "ff", base=16}        # → 255
parse{Int, "1010", base=2}       # → 10

# 5. Комплексные числа
parse{Complex{Float64}, "1+2im"} # → 1.0 + 2.0im

# 6. Ошибка при неверном вводе
try
  parse{Int, "abc"}
catch e
  println("Ошибка: ", e)
end

# 7. tryparse – безопасный вариант
tryparse{Int, "abc"}              # → nothing
tryparse{Int, "42"}               # → 42

Ошибка: ArgumentError("invalid base 10 digit 'a' in \"abc")

[23]: 42

[24]: println("Сложение:      2 + 3      = ", 2 + 3)
      println("Вычитание:     5 - 2      = ", 5 - 2)
      println("Умножение:     4 * 3      = ", 4 * 3)
      println("Деление:       10 / 3     = ", 10 / 3)
      println("Целочисл. дел.: 10 ÷ 3     = ", 10 ÷ 3)
      println("Остаток:       10 % 3     = ", 10 % 3)
      println("Степень:       2 ^ 10     = ", 2 ^ 10)
      println("Квадр. корень:  sqrt(16)   = ", sqrt(16))
      println("Куб. корень:   cbt(27)    = ", cbrt(27))
      println("Модуль:        abs(-7)    = ", abs(-7))

Сложение:      2 + 3      = 5
Вычитание:     5 - 2      = 3
Умножение:     4 * 3      = 12
Деление:       10 / 3     = 3.3333333333333335
Целочисл. дел.: 10 ÷ 3     = 3
Остаток:       10 % 3     = 1
Степень:       2 ^ 10     = 1024
Квадр. корень: sqrt(16)   = 4.0
Куб. корень:   cbrt(27)   = 3.0
Модуль:        abs(-7)    = 7
```

4.3.3 Задание 3. Математические операции

С переменными $a = 10$ и $b = 3$ выполнены: - арифметические операции (+, -, *, /, div, rem, ^); - вычисление квадратного и кубического корня (sqrt, cbrt); - операции сравнения (==, !=, >, <, >=, <=); - логические операции (&&, ||, !).

```
[25]: a = 5          # Int64
      b = 2.5      # Float64
      c = 3 + 4im  # Complex{Int64}

      println("a + b = ", a + b, " тип: ", typeof(a + b))
      println("a * c = ", a * c, " тип: ", typeof(a * c))
      println("b / c = ", b / c, " тип: ", typeof(b / c))
      println("2^0.5 = ", 2^0.5, " тип: ", typeof(2^0.5))

      a + b = 7.5 тип: Float64
      a * c = 15 + 20im тип: Complex{Int64}
      b / c = 0.3 - 0.4im тип: ComplexF64
      2^0.5 = 1.4142135623730951 тип: Float64

[26]: println("1 == 1.0 → ", 1 == 1.0) # true (значения равны)
      println("1 === 1.0 → ", 1 === 1.0) # false (разные типы)
      println("1 != 2 → ", 1 != 2) # true
      println("3 > 2 → ", 3 > 2) # true
      println("3 < 2 → ", 3 < 2) # false
      println("3 >= 3 → ", 3 >= 3) # true
      println("3 <= 2 → ", 3 <= 2) # false
      println("NaN == NaN → ", NaN == NaN) # false
      println("NaN != NaN → ", NaN != NaN) # true

      1 == 1.0 → true
      1 === 1.0 → false
      1 != 2 → true
      3 > 2 → true
      3 < 2 → false
      3 >= 3 → true
      3 <= 2 → false
      NaN == NaN → false
      NaN != NaN → true

[27]: println("true && false → ", true && false) # false
      println("true || false → ", true || false) # true
      println("!true → ", !true) # false
      println("xor(true,false) → ", xor(true, false)) # true

      # Короткое замыкание
      x = 5
      println("x > 0 && x < 10 → ", x > 0 && x < 10) # true
      println("x < 0 || x > 3 → ", x < 0 || x > 3) # true
```

Рисунок 4.4: Математические операции

4.3.4 Задание 4. Операции с матрицами и векторами

Подключён пакет LinearAlgebra. Созданы матрицы A, B и вектор v. Выполнены:

- сложение и вычитание матриц;
- умножение на скаляр;
- матричное умножение ($A * B$);
- транспонирование (A');
- скалярное произведение (dot);
- поэлементные операции ($.*$, $./$, $.^$);
- умножение матрицы на вектор

```

true && false → false
true || false → true
!true → false
xor(true, false) → true
x > 0 && x < 10 → true
x < 0 || x > 3 → true

```

[28]:

```

A = [1 2; 3 4]
B = [5 6; 7 8]

println("A = ", A)
println("B = ", B)
println("A + B = ", A + B)
println("A - B = ", A - B)

A = [1 2; 3 4]
B = [5 6; 7 8]
A + B = [6 8; 10 12]
A - B = [-4 -4; -4 -4]

```

[29]:

```

println("2 * A = ", 2 * A)
println("A * 3 = ", A * 3)
println("A / 2 = ", A / 2)

2 * A = [2 4; 6 8]
A * 3 = [3 6; 9 12]
A / 2 = [0.5 1.0; 1.5 2.0]

```

[30]:

```

println("A * B = ", A * B)
v = [1, 2]
println("A * v = ", A * v)

A * B = [19 22; 43 50]
A * v = [5, 11]

```

[31]:

```

println("A' = ", A')      # транспонирование
println("v' = ", v')      # вектор-столбец → вектор-строка

A' = [1 3; 2 4]
v' = [1 2]

```

[32]:

```

v = [1, 2, 3]
w = [4, 5, 6]

# Способ 1: через транспонирование
println("v' * w = ", v' * w)

```

Рисунок 4.5: Операции с матрицами и векторами

5 Результаты

- Установлены Julia (версия 1.12.5) и Jupyter Lab (версия 4.6.3).
- Установлен пакет IPython для работы Julia в Jupyter.
- В Jupyter Lab создан ноутбук с ядром Julia.
- Успешно выполнены все примеры и задания.
- Получены корректные результаты вычислений, включая работу с матрицами и векторами.
- Все скриншоты зафиксированы и приложены к отчёту.

6 Обсуждение результатов

Все задания выполнены без ошибок. Установка через `snar` оказалась удобным решением, так как пакет `julia` отсутствовал в стандартных репозиториях `apt`. Пакет Julia установился без проблем и корректно зарегистрировался как ядро для Jupyter.

7 Заключение

В ходе лабораторной работы были выполнены все поставленные задачи.

Цель работы достигнута. В дальнейшем планируется изучение более сложных пакетов Julia для статистического анализа и визуализации данных.

Список литературы

1. IJulia GitHub. — URL: <https://github.com/JuliaLang/IJulia.jl>.
2. Julia Documentation. — 2020. — URL: <https://docs.julialang.org/>.
3. Jupyter Lab Documentation. — URL: <https://jupyterlab.readthedocs.io/>.
4. *Newham C.* Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 p. — (In a Nutshell). — ISBN 0596009658. — URL: <http://www.amazon.com/Learning-bash-Shell-Programming-Nutshell/dp/0596009658>.
5. *Robbins A.* Bash Pocket Reference. — O'Reilly Media, 2016. — 156 p. — ISBN 978-1491941591.
6. *Zarrelli G.* Mastering Bash. — Packt Publishing, 2017. — 502 p. — ISBN 9781784396879.
7. *Таненбаум Э., Бос Х.* Современные операционные системы.. — 4-е изд. — СПб. : Питер, 2015.. — 1120 с. — (Классика Computer Science).